# Cahier des Charges:
# CASTLE_MAZE

## I. Présentation:

- Qui ?

  BERNARD Yohan, numéro d'étudiant : 12410425
  CHAIBI Chaouki, numéro d'étudiant : 10916173
  JOLIVALT Mathieu, numéro d'étudiant : 12408245

- Contexte :

  Un projet universitaire visant à développer un jeu qui consiste en un roguelike dans un univers de médieval-fantaisie.
  Dans un groupe de 3 personnes, nous allons créer ce jeu en langage C++ en utilisant des bibliothèques telles que SDL.

## II. Description du projet:

- Résultat visé :

  Le jeu commence par un écran titre, suivi d'un écran de sélection de niveau. Le joueur apparaît ensuite dans le village, où il peut interagir avec des PNJ et obtenir des missions. Il explore ensuite différentes zones de combat contenant des mobs, des boss et des coffres. Les combats permettent de récupérer du loot, d'améliorer les statistiques du personnage et d'augmenter le score. Le niveau se termine lorsque le boss principal du donjon est vaincu. Le score final dépend des monstres tués, des objets récupérés, des pièces collectées et des PNJ sauvés.

- Fonctionnalités :

  ➔ Lancement du jeu:

  Le jeu démarre sur un écran titre.

  Fonctionnalités :

  - afficher le titre du jeu
  - afficher le bouton Jouer
  - afficher le bouton Quitter

  ➔ Sélection du niveau:

  Quand le joueur clique sur **Jouer**, il arrive sur l'écran de sélection de niveau.

Fonctionnalités :

- afficher la liste des niveaux
- permettre de choisir un niveau
- afficher éventuellement le meilleur score du niveau

➔ Initialisation de la partie:

Une fois le personnage choisi, la partie commence.

Fonctionnalités :

- créer le joueur
- charger le niveau
- initialiser l'inventaire
- initialiser le score
- placer le joueur dans le village

➔ Écran village:

Le joueur arrive dans le village.

Fonctionnalités :

- afficher le village
- déplacer le joueur dans le village
- afficher les PNJ
- parler avec les PNJ
- récupérer des informations ou des missions
- permettre l'accès à la sortie du village ou au portail

➔ Exploration des zones:

Fonctionnalités :

- charger une zone
- afficher les ennemis, coffres et PNJ éventuels
- déplacer le joueur dans la zone
- passer d'une zone à une autre

➔ Apparition des ennemis:

Dans les zones de combat, des ennemis apparaissent.

Fonctionnalités :

- générer des mobs

- donner des statistiques à chaque ennemi
- gérer leur comportement de base
- faire apparaître un boss dans certaines zones

➔ Combat:

Le joueur peut combattre les ennemis.

Fonctionnalités :

- attaquer un ennemi
- esquiver les attaques ennemies
- calculer les dégâts
- retirer des points de vie
- vérifier si une entité est morte

➔ Boss:

Certaines zones ou le donjon contiennent un boss.

Fonctionnalités :

- faire apparaître un boss
- lui donner plus de points de vie et des attaques spéciales
- déclencher le combat contre le boss
- vérifier sa défaite

➔ Coffres:

Le joueur peut trouver des coffres dans les zones.

Fonctionnalités :

- afficher les coffres
- permettre leur ouverture
- générer leur contenu

Contenu possible :

- pièces
- objets
- bonus de statistiques

➔ Loot et objets:

Les mobs, boss et coffres peuvent donner des récompenses.

Fonctionnalités :

- récupérer le loot
- ajouter les objets dans l'inventaire
- ajouter les pièces au score
- appliquer les bonus des objets au joueur

➔ <u>Inventaire:</u>

Le joueur possède un inventaire.

Fonctionnalités :

- stocker les objets récupérés
- afficher la liste des objets
- utiliser un objet
- supprimer un objet utilisé

➔ <u>Gestion des statistiques:</u>

Les statistiques du joueur changent selon les objets ou les combats.

Fonctionnalités :

- gérer les PV
- gérer l'attaque
- gérer la défense
- modifier les statistiques avec les objets
- vérifier les limites éventuelles

➔ <u>PNJ en danger:</u>

Certains PNJ peuvent être en danger.

Fonctionnalités :

- marquer un PNJ comme en danger
- permettre au joueur de le sauver
- mettre à jour son état
- ajouter des points au score si le PNJ est sauvé

➔ <u>Score:</u>

Le jeu calcule un score pour chaque niveau.

Fonctionnalités :

- compter les monstres tués
- compter les objets récupérés

- compter les PNJ sauvés
- compter les pièces collectées
- calculer le score final

➔ <u>Fin du niveau:</u>

Le niveau se termine quand le boss principal est vaincu.

Fonctionnalités :

- vérifier la mort du boss final
- terminer le niveau
- afficher le score final
- enregistrer le meilleur score

➔ <u>Défaite du joueur:</u>

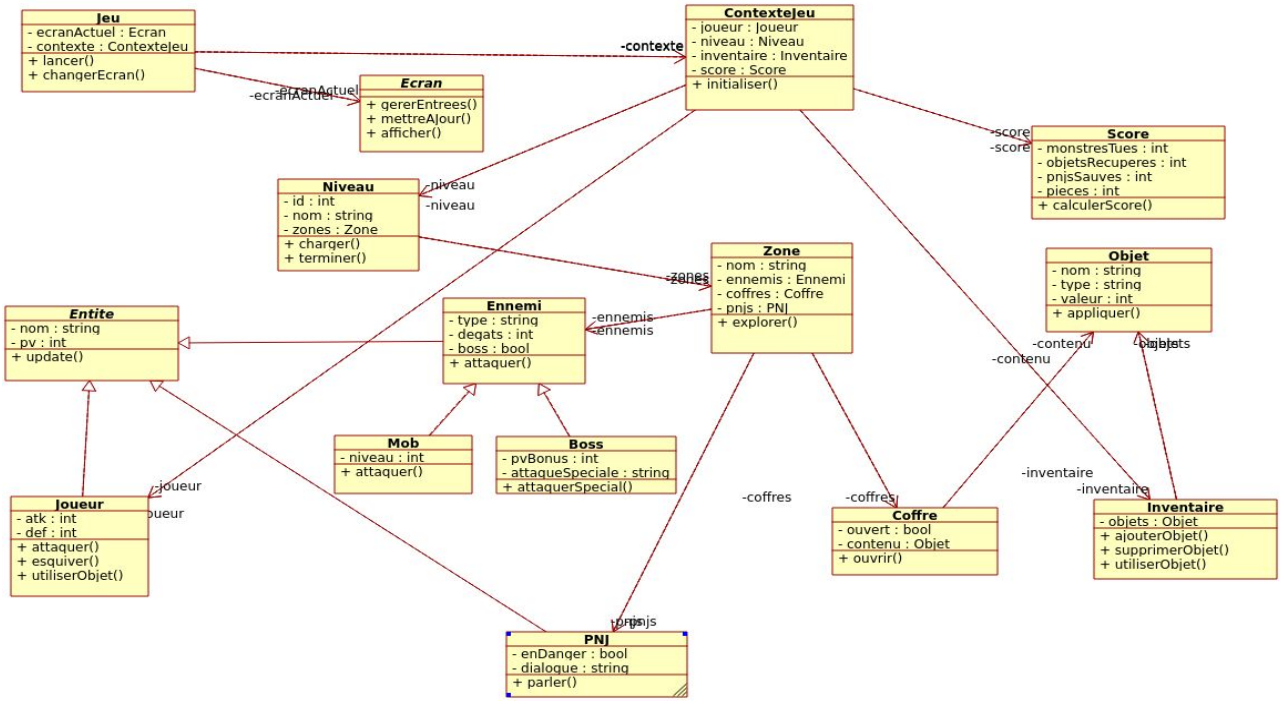
Le joueur peut perdre la partie.

Fonctionnalités :

- détecter la mort du joueur
- afficher un écran de défaite ou de fin
- proposer de recommencer.

## III. <u>Diagramme des classes:</u>

➔ Le diagramme a été fait avec UMBRELLO.



## IV. Contraintes:

1) Contraintes techniques:

- Compatibilité multi-plateforme : le jeu doit fonctionner de manière fluide sur Windows, Linux et iOS.
- Optimisation : Maintenir des performances élevées malgré la génération aléatoire et le nombre d'éléments à l'écran.
- Support de contrôle : compatibilité avec clavier/souris sans latence.

2) Contraintes artistiques :

- Cohérence visuelle : Maintenir une direction artistique homogène.
- Animations fluides : Assurer des animations réactives pour une bonne lisibilité.
- Ambiance sonore : Créer une atmosphère immersive et dynamique.

3) Contraintes temporelles :

- Durée de développement : 6 semaines, nécessitant une gestion stricte des priorités.
- Phase de test : Débogage rapide et efficace pour livrer un produit stable.

## V. Le déroulement du projet:

- Tâche 1 :
  Conception et Initialisation, Rédaction du cahier des charges et création du Dépôt Git commun.
  Conception du diagramme de classes via Umbrello et élaboration du diagramme de Gantt.
  Livrable : Cahier des charges finalisé rendu sur TOMUSS.

- Tâche 2 :
  Moteur Graphique et Structure, Implémentation de la bibliothèque graphique SDL
  Création des classes Jeu, ContexteJeu et Ecran (intégrant les méthodes gererEntrees(), mettreAjour(), afficher()) , creation du makefile .

- Tâche 3 :
  Système de Combat et Entités
  Développement de la classe mère Entite (variables nom, pv, et méthode update()).
  Implémentation de la classe Joueur (attaquer(), esquiver()) et de la classe Ennemi (degats), associées au support clavier/souris sans latence.
  Livrable : Démo mi-parcours (Moteur de combat fonctionnel).
  Dépendance : Tâche 2 validée (nécessite l'affichage Ecran).

- Tâche 4 :
  Exploration et Économie
  Génération des classes Niveau et Zone pour cartographier les batailles.
  Développement du système de loot aléatoire : interaction avec la classe Coffre (ouvert, contenu), application des modificateurs de la classe Objet (valeur), et gestion de la classe Inventaire (ajouterObjet(), supprimerObjet()).

Dépendance : Tâche 3 validée.

- Tâche 5 :
  Enjeux et Immersion
  Création de la classe Boss héritant d'Ennemi, intégrant les variables pvBonus et la méthode attaqueSpeci.
  Intégration de la classe PNJ dans le village avec gestion des boîtes de dialogue et du booléen enDanger.
  Ajout de l'ambiance sonore pour créer une atmosphère immersive et dynamique.
  Dépendance : Tâches 3 et 4 validées.

- Tâche 6 :
  Règles et Fin de Partie
  Implémentation de la classe Score calculant les points finaux via les variables monstresTues, objetsRecuperes, pieces, et pnjsSauves.

  Dépendance : Tâches 4 et 5 validées.

- Tâche 7 :
  Phase de Tests et Optimisation
  Débogage et Test de toutes les fonctionnalitéset Optimisation si nécéssaire .
  Dépendance : Intégration continue requise tout au long des Tâches 2 à 6.

- Tâche 8 :
  Préparation de la Soutenance
  Vérification finale des livrables et du fonctionnement fluide de l'exécutable.
  Livrable : Soutenance de projet devant le jury.

- Le diagramme de Gantt :

| Phase | Tâche | Semaine 1 (03/03) | Semaine 2 (09/03) Rendu Cahier des Charges TOMUSS | Semaine 3 (17/03) Demo-Mi Parcour | Semaine 4 (24/03) | Semaine 5 (31/03) | Semaine 6 (07/04) | Semaine 7 (14/04) | Semaine 8 (21/04) | Semaine 9 (28/04) Rendu Final & Présentation |
|---|---|---|---|---|---|---|---|---|---|---|
| Tâche 1 : Conception et Initialisation | Creation du depot GitForge | | | | | | | | | |
| Tâche 1 : Conception et Initialisation | Rédaction et complétion du cahier des charge | | | | | | | | | |
| Tâche 1 : Conception et Initialisation | Conception du diagramme de classes | | | | | | | | | |
| Tâche 1 : Conception et Initialisation | élaboration du diagramme de Gantt. | | | | | | | | | |
| Tâche 2 : Moteur Graphique et Structure | Implémentation de la bibliothèque graphique SDL | | | | | | | | | |
| Tâche 2 : Moteur Graphique et Structure | Création et implémentation de la classe Ecran. | | | | | | | | | |
| Tâche 2 : Moteur Graphique et Structure | Création & Implémentation de la classe ContexteJeu | | | | | | | | | |
| Tâche 2 : Moteur Graphique et Structure | Creation & complétion du Makefile | | | | | | | | | |

| Phase | Tâche | Semaine 1 (03/03) | Semaine 2 (09/03) Rendu Cahier des Charges TOMUSS | Semaine 3 (17/03) Demo-Mi Parcour | Semaine 4 (24/03) | Semaine 5 (31/03) | Semaine 6 (07/04) | Semaine 7 (14/04) | Semaine 8 (21/04) | Semaine 9 (28/04) Rendu Final & Présentation |
|---|---|---|---|---|---|---|---|---|---|---|
| Tâche 2 : Moteur Graphique et Structure | Création & implémentation de la classe Jeu | | | | | | | | | |
| Tâche 3 : Système de Combat et Entités | Définition et Implémentation de la classe mère Entite. | | | | | | | | | |
| Tâche 3 : Système de Combat et Entités | Implémentation de la classe Joueur | | | | | | | | | |
| Tâche 4 : Exploration et Économie | Génération des classes Niveau et Zone (cartographie). | | | | | | | | | |
| Tâche 4 : Exploration et Économie | Développement du système de loot (Coffre, Objet, Inventaire). | | | | | | | | | |
| Tâche 5 : Enjeux et Immersion | Création & implémentation de la classe Boss. | | | | | | | | | |
| Tâche 5 : Enjeux et Immersion | Intégration de la classe PNJ (boîtes de dialogue). | | | | | | | | | |

| Phase | Tâche | Semaine 1 (03/03) | Semaine 2 (09/03) Rendu Cahier des Charges TOMUSS | Semaine 3 (17/03) Demo-Mi Parcour | Semaine 4 (24/03) | Semaine 5 (31/03) | Semaine 6 (07/04) | Semaine 7 (14/04) | Semaine 8 (21/04) | Semaine 9 (28/04) Rendu Final & Présentation |
|---|---|---|---|---|---|---|---|---|---|---|
| Tâche 5 : Enjeux et Immersion | Ajout de l'ambiance sonore avec SDL, | | | | | | | | | |
| Tâche 6 : Règles et Fin de Partie | Implémentation de la classe Score. | | | | | | | | | |
| Tâche 7 : Phase de Tests et Optimisation | Débogage et test des fonctionnalités, | | | | | | | | | |
| Tâche 7 : Phase de Tests et Optimisation | Generation de la documentation avec doxygen, verification des commentaires . | | | | | | | | | |
| Tâche 7 : Phase de Tests et Optimisation | Optimisation du code source (performances). | | | | | | | | | |
| Tâche 8 : Préparation de la Soutenance | Vérification finale des livrables et du fonctionnement fluide de l'exécutable. | | | | | | | | | |
| Tâche 8 : Préparation de la Soutenance | Soutenance de projet devant le jury. | | | | | | | | | |